

面试押题 07 | Mooncake：存储与传输系统正确性

对应简历「Mooncake（7 个 PR 已合入）」。核验日期：2026-09-29；作者账号 XFDG。以下以 GitHub 最终合入 diff 为准，开发记录中的早期方案只用于解释演进，不当作最终实现。本文整理既有证据，本轮未重跑硬件测试。

1. 仓库做什么，我贡献在哪里

Mooncake 面向大模型推理的 KVCache 存储与高性能数据传输：Store 管理对象、副本、内存与磁盘层；Transfer Engine / TENT 提供 RDMA 等传输能力。它为分离式推理和跨实例缓存复用提供基础设施，但我没有独立开发整个调度平台。

简单技术原理：上层推理引擎把可复用 KV cache 作为对象保存；Store 维护对象位置、可用副本和生命周期，Transfer Engine 负责跨设备/节点传输。RDMA 降低数据搬运中的 CPU 参与，但内存注册、连接状态与失败处理仍要由软件正确管理。

典型应用：Prefill/Decode 分离时把 prefill 产出的 KV 传给 decode；跨实例复用已有前缀；将冷 KV 下沉到内存或磁盘，减轻显存压力。这是缓存与数据传输基础设施，不替代 attention 计算。应用背景见 [官方介绍](#)。

我的贡献是修复 Store 和传输层的具体错误路径，重点是「数据何时可见、资源何时销毁、失败如何向上传递」，并补充能够命中问题分支的回归测试。

PR	最终合入范围	合入日期（UTC）
#3601	Bucket 数据先落盘再提交元数据，失败清理孤儿文件	2026-08-31
#3604	RDMA 端口恢复时淘汰失效 endpoint	2026-08-27
#3660	注销 MR 后恢复内存 fork 属性	2026-08-26
#3726	offload 没有入队时不能返回成功	2026-09-02
#3662	本地 MEMORY 优先级不受副本顺序影响	2026-09-21
#4278	BatchReplicaClear RPC 保留租户信息	2026-09-23
#4016	隔离 Linux hugepage 依赖，修复 Store 构建兼容	2026-09-23

2. 60 秒 STAR 口述

S：大模型推理缓存不仅需要快，存储、传输和多租户错误路径也会影响长期可用性。**T：**我针对 Mooncake 的具体 issue 做根因定位、最小修改和回归验证。**A：**沿 Store 的写入/副本/RPC 路径以及 RDMA 的 endpoint/MR 生命周期核对不变式；例如补齐数据落盘屏障、恢复端口时重建 endpoint、把租户参数贯穿 RPC，并根据 review 修正测试覆盖不足的问题。**R：**目前 7 个 PR 已合入，结果主要是持久性、故障恢复和隔离语义的修复，不把 issue 中的生产异常数字说成我测得的吞吐提升。

3. 每个 PR 怎么讲

3.1 #3601：元数据存在，不等于数据已经持久化

- **问题：**WriteBucket() 的实际写路径拿到 PosixFile，原来仅在 UringFile 分支执行的 datasync() 没覆盖它；元数据可能先提交，异常退出后出现元数据指向未稳定落盘数据的风险。

- **我的思路:** 先核对 `OpenFile()` 返回的真实文件实现, 区分「write 返回」「page cache 可读」和「持久化完成」。真正需要守住的是数据同步成功先于元数据提交。
- **最终方案:** 在两个写分支之后统一调用虚函数 `file->datasync()`, 成功后再清缓存并写元数据; 同步失败返回 `FILE_WRITE_FAIL`, 调用 `CleanupOrphanedBucket()`, 避免重试不断遗留 `.bucket` 文件。
- **结果与证据:** 已合并; 新增 `DatasyncFailureRemovesOrphanBucketFile`, 注入同步失败后断言错误码、无 `.bucket`、无 `.meta`、key 不可查询。本地历史记录含 C++ 编译、wheel 构建/import 和 CI 记录, 但不是断电实验, 不能表述为完成所有崩溃一致性验证。最终 diff

一句话复述: 我补的是「数据先稳定落盘、元数据后提交」这个提交顺序, 同时补齐同步失败后的清理。

3.2 #3604: 端口恢复不代表旧 QP 恢复

- **问题:** 端口 down/up 后, `resume()` 原来只切换设备状态, 缓存中的 QP 可能已进入 ERR; 后续请求继续取旧 endpoint, 导致停滞或超时。
- **我的思路:** 把物理链路状态和软件连接对象生命周期分开。恢复链路后要使旧连接失效, 并复用已有按需建连逻辑。
- **最终方案:** 补 `EndpointStore::evictAll()`, 覆盖 FIFO/SIEVE 两种实现; 在锁内把 endpoint 发布到 waiting list 并清 active map, 锁外调用 `beginDestroy()`, 避免回调 `reclaim()` 重入同一把锁; 成功从 PAUSED 恢复后触发淘汰, 下次访问建立新 endpoint。
- **结果与证据:** 2026-08-27 合并。PR 记录了状态路径核对和 diff review, 没有本人的硬件端口 flap A/B 性能实测。issue 的 5.48 s / 1.03 s 是报告者的异常现象, 不能写成我的优化收益。最终 diff

一句话复述: 我让恢复动作同步失效旧 endpoint, 并把缓存锁和底层资源销毁解耦。

3.3 #3660: 显存/内存没用满, 也可能 ENOMEM

- **问题:** 目标环境 RDMA fork protection 下, 注册过的地址区间保留 `MADV_DONTFORK` 属性; 频繁注册/注销后 VMA 分裂不易合并, 可能先耗尽映射数量而不是物理内存。
- **我的思路:** 同时检查 MR 资源和 OS 虚拟内存属性的生命周期。注销不能只考虑 `ibv_dereg_mr()`, 还要恢复该路径附加的状态。
- **最终方案:** 在传统 TE 与 TENT 注销路径中, 先缓存地址和长度, 再调用 `dereg`; 只有成功后才执行 `madvise(..., MADV_DOFORK)`。提前缓存是为避免注销后读取已释放的 `ibv_mr`; 恢复属性失败记录 warning, 不伪装成 MR 仍未注销。
- **结果与证据:** 2026-08-26 合并。PR 记录北京 H200 环境 TE 编译通过; 历史 Store 测试为 98/99, 1 项已有失败, 因此不能说「全套测试全绿」; 没有长期 register/unregister 压测数字。最终 diff

边界: 这是对应 issue 和内存注册约定下的修复, 不能推广为所有 libibverbs/provider 都有同一缺陷; 若讨论重叠注册区间, 仍需核对 provider 引用计数与地址范围管理。

3.4 #3726: 没有入队, 就不能让上层记一笔成功任务

- **问题:** `PushOffloadingQueue()` 在 segment 列表为空或全部为 `nullopt` 时没有提交工作却返回成功, 调用者随后增加 refcount、登记 offloading task, 形成 phantom task。
- **我的思路:** 从返回值追到三个调用点, 确认它们把成功当作「确实提交了任务」; 因此修复返回契约比额外清理幽灵任务更直接。
- **最终方案:** 空列表直接返回 `UNABLE_OFFLOADING`; 循环用 `any_enqueued` 记录实际入队, 全部跳过也返回失败。三个调用点已有成功条件保护, 无需重复增加补丁。
- **结果与证据:** 2026-09-02 合并; 最终新增 `PushOffloadingQueueReportsNoopAsFailure`, 直接构造空 segment 与全 `nullopt` 两类退化副本并命中相应分支。
- **review 中的关键教训:** 早期 public-path 测试只走了原本就会失败的 `SEGMENT_NOT_FOUND`, 补丁前也能过; 后来改成 friend fixture helper 精确测试私有分支。C++ 的 friendship 不被 `TEST_F` 子类继承, 因此私有类型构造及函数调用放在 fixture 成员中。最终 diff

边界: 只解决 silent-success/phantom-task 部分; 不是修复全部 offload 问题, 也不是 RDMA MPSC 队列满活锁。后者是另一条 #3661, 不能计入这 7 个已合入 PR。

3.5 #3662: 选「最优副本」, 不能选「先遇到的本地副本」

- **问题:** 契约规定 local MEMORY 优先于 local NOF, 但原实现遇到任一本地副本就返回; `[NOF, MEMORY]` 和 `[MEMORY, NOF]` 得到不同结果。
- **我的思路:** 把遍历候选与应用优先级分开, 消除 master 返回顺序对策略的影响, 同时保留可读状态约束。
- **最终方案:** 完整扫描可读 MEMORY/NOF 候选, 分别记录本地/远端, 再按既有优先级决策。其他存储层级和远端评分逻辑不改。
- **结果与证据:** 2026-09-21 合入; 三项回归覆盖顺序交换、incomplete MEMORY 回退 NOF、本地 NOF 仍优先于远端 MEMORY。后两项 guard case 由另一位贡献者提交改编而来, 已在 PR 中注明协作来源; 最终收敛后没有额外本地运行, 不能把早期测试冒充最终 head 验证。最终 diff

特别注意: 最终合入版不再改 `batch_get_session_start`, 不宣称支持 SSD/NOF ranged-session, 也不宣称彻底解决 #3658。旧开发文档保留了早期方案, 面试以此处为准。

3.6 #4278: 租户在客户端存在, 却在 RPC 边界丢了

- **问题:** `MasterClient::BatchReplicaClear` 没把客户端 tenant 传给 RPC, 服务端按 default tenant 执行; 同名 key 可能清错租户而目标租户数据未清。
- **我的思路:** 不仅检查 service 有无 tenant 参数, 还沿 client → RPC wrapper → tenant-aware service 全链路核对。必须用两个租户的同名 key 才能揭示隔离错误。
- **最终方案:** RPC 传递 `tenant_id`, wrapper 使用 `WithRequestTenant` 解析校验, 转到已有 tenant-aware overload; 关闭租户配额时保留 default 行为。
- **结果与证据:** 2026-09-23 合入; 同进程真实 RPC 回归在 baseline 7364f79 上失败, 在 fixed 构建通过, `batch_replica_clear_tenant_rpc_test` 为 1/1。测试先清 tenant-a, 确认 default 同名 key 保留, 再验证 default 自己可正常清理。最终 diff

一句话复述: 我修复了 RPC 参数遗漏导致的跨租户清理错误, 并用双租户同名对象建立修前失败、修后通过的证据。

3.7 #4016: 平台特性不要通过公共头污染所有编译单元

- **问题:** Store 公共头无条件引入 Linux hugepage 头, 破坏非 Linux 构建; 中间修订又把 helper 挪进头文件, 产生声明依赖与重复定义问题。
- **我的思路:** 收窄为 Store hugepage 平台隔离, 不把 macOS 全部 transport、wheel 打包一起改。实现应保留单一定义, 平台条件只包围 Linux 专用部分。
- **最终方案:** 用 `__linux__` 保护头文件与 hugepage flags, helper 仍在 `.cpp` 定义; 非 Linux 保留请求尺寸、不修改 flags 并告警; 补上 `WITH_STORE=ON/WITH_TE=OFF` 的 TE 头搜索路径。
- **结果与证据:** 2026-09-23 合入; 记录包含两条预处理分支 header smoke compile、B200 上默认 TE+Store、`USE_CUDA=OFF` 构建 226/226 targets。TE 关闭模式只验证 Store 对象越过原失败点, 最终链接仍依赖外部 TE 库; 不能说「macOS 完整运行栈已支持」。最终 diff

4. 高频追问与具体答法

Q1. 为什么 write 成功还要 datasync?

write 成功通常说明数据已被内核接收, 不等于介质持久化。这个提交协议要求元数据发布前数据同步成功; 失败就不提交元数据, 并清掉不能作为有效对象使用的文件。本 PR 没声称涵盖目录项持久化等全部断电模型。

Q2. #3601 的注入测试是不是证明了真实断电安全？

不是。它证明同步失败分支返回错误、清文件、拒绝发布元数据；真实断电恢复还需故障注入/重启与文件系统语义验证。不同测试的证明范围不能混用。

Q3. RDMA link up 后为什么不能继续用旧连接？

物理端口状态和 QP 状态独立。进入 ERR 的 QP 不会因为 link up 自动回到正常收发状态；要淘汰旧 endpoint，让后续访问重新走连接建立。

Q4. 为什么 waiting list 在锁内改、beginDestroy 在锁外调用？

waiting list 与 active map 的一致性需要同一把锁保护；但销毁过程可能回调 reclaim 并再次获取该锁，锁内调用会有重入死锁风险。先在锁内转移所有权，再锁外执行动作。

Q5. ENOMEM 除了物理内存不足还有什么解释？

VMA 数量、锁页限制、设备注册资源等都可能形成约束。#3660 聚焦目标 issue 的 VMA 分裂，诊断时可以看 `proc/<pid>/maps` 数量、`vm.max_map_count` 和注册/注销前后变化，不能只看剩余内存。

Q6. 为什么注销 MR 前先取地址和长度？

注销成功后 MR 对象可能已释放，再读取字段就是 use-after-free。把恢复 OS 属性所需信息提前保存，且只有 dereg 成功后才执行恢复动作。

Q7. 如何判断回归测试真的有用？

先在旧实现跑，必须命中修复分支并失败，再在新实现通过。#3726 初版测试两边都过就是无效证据，后来直接构造两类退化状态，测试才真正约束返回值。

Q8. 把 local MEMORY 放到列表第一项不就行了吗？

不能把正确性寄托在上游返回顺序。selector 的接口允许任意顺序，应扫描完再应用优先级；还要跳过未完成副本，不能为了内存优先选中尚不可读的数据。

Q9. 多租户问题为什么用 RPC 测试而不是直接调用 service？

service 本来就有 tenant-aware overload，直接调它会绕过参数丢失点。真实 RPC 才覆盖 client 序列化、wrapper 参数与 service 转发，同名 key 则验证不会跨租户误清。

Q10. 这 7 个 PR 带来了多少性能提升？

不能合并报一个性能百分比。它们主要修复正确性和恢复行为，证据是定向测试、编译及合入状态。别人的 issue 异常数据不是我自己的 A/B 实验，build targets 数也不是性能 case 数。

Q11. 怎么体现个人贡献，而不是只改几行？

具体讲根因链、分支约束、测试设计和 review 收敛。例如 #4278 是全 RPC 链路定位与修前修后对照，#3726 是发现原测试没有覆盖补丁；行数少不妨碍问题有系统性。

Q12. 哪个地方是你与维护者协作完成的？

PR #3662 与已合入 session 工作去重后收窄为顺序无关选择，迁入另一贡献者的 guard case 并保留来源；#3601 根据 review 补齐失败清理。这些属于协作交付，不说所有测试和原始方案都由自己独立提出。

5. 证据入口与表达边界

· GitHub: 各小节 PR 正文、review 与最终 diff; 状态已于 2026-09-29 经 API 的 `merged_at` 核对。

- 北京开发记录: B200 OSS 修复记录、offload 返回值迭代记录、#3601 核验记录。
- 租户修复: H200 开源排障记录。
- 状态与实现以最终 `merged_at` / diff 为准; 不计 #3661 等开放或历史关闭 PR; 合入不等于线上收益已验证。